

Lecture 5: Expectations — Transition Dynamics

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 15, 2026

University of Toronto

HouseholdStages

HouseholdStages

- An implementation of Stages for building household blocks of heterogeneous-agent macro models.
- Auclert–Bardóczy–Rognlie–Straub **SSJ toolkit** (Python) also has stage implementations.
- Econ-ARK also has a form of Stages.

What the package gives you

- **Stage implementations.** MarkovStage, WealthChangeStage, ConsumptionSavingsStage, LogitChoiceStage, MigrationStage, BorrowingConstraintStage, AssetPriceChangeStage ...
- **Composition.** ○ chains Stages; lift_moments attaches integrals.
- **Inner-solve helpers.** Steady state and transition V , Λ given env (prices and params).
- **Functorial lifts.** lift_jacobian (auto-diff), lift_gpu.
- **SSJ utilities.** Sequence-space Jacobian (covered in L6+).

What the package does *not* do

The library handles **the household block at a given env.** It does *not*:

- Decide your aggregate state.
- Compute prices from $\bar{K}$ (firm block).
- Run outer loops for calibration or market clearing.

Why this split

- Household block: **generic components** across HA models.
- Equilibrium, calibration, transition: **model-specific**.

Steady-state inner-solve helpers

```
# V backward to a fixed point at fixed env.  
solve_vfi_steady_state_given_env!(hh, env)  
  
#  $\Lambda$  forward to stationarity using the policy populated by the backward sweep.  
solve_lambda_steady_state_given_env!(hh)  
  
# Bundle of the two: V then  $\Lambda$ , plus attached moments.  
solve_steady_state_given_env!(hh, env)
```

All three take env **as given**. They iterate the household block but do not update prices or parameters.

Aiyagari Steady State in Stages

Same Model as Before

- **Stage Chain** (time order):

$z_shock \circ_s income \circ_s savings.$

- **Firm.** Cobb-Douglas, fixed labor L .
- **Equilibrium.** $\bar{K} = \bar{K}^{\text{supplied}}(\bar{K})$.
- **Outer loop.** Damped **tatonnement** on $\bar{K}$.

State Layout

```
layout = StateLayout(  
    StateAxis(:wealth, continuous_grid(0, 100; length=400, spacing=:log)),  
    StateAxis(:z,      [0.6, 1.0, 1.4]),  
)
```

- **wealth** (continuous, 400-pt log grid).
- **z** (productivity; discrete, 3 levels).
- Closures read `cell.wealth`, `cell.z`.

Stage 1: z Shock

```
z_shock = MarkovStage(layout; axis=:z, transition=P_z)
```

Stage 2: Income

```
income = WealthChangeStage(layout;  
    wealth_post = (cell; env) ->  
        (1 + env.r) * cell.wealth + env.w * cell.z,  
)
```

- Previous example snapped-to-grid. The library function reinterpolates continuously.

Stage 3: Consumption Savings

```
savings = ConsumptionSavingsStage(layout; β=0.96,  
    utility=(cell, c; env) -> c < 0 ? -Inf : log(c),  
    monotone_search=:divide_conquer,  
)
```

- :divide_conquer: faster $O(N \log N)$ algorithm for when policy is monotone in b .

Compose

```
hh = z_shock ◦ income ◦ savings
```

- ◦ composes Stages in time order.
- The household block is *itself a Stage*: own backward! and forward!.

Define Moments

```
hh = define_moments!(hh;  
    K_supplied=at_end(integrand=:wealth, reduce=sum),  
)
```

- `define_moments!`: Define integrals on Λ . Here $K^{\text{supplied}} = \int b d\Lambda^{\text{end}}$.

Solve Steady State Given $\bar{K}$

```
env = make_env(hh; aiyagari_prices(K, p)...)      # (r, w)
res = solve_steady_state_given_env!(hh, env)
K_supplied = res.moments.K_supplied
```

- **Line 1.** Build env: (K, r, w) .
- **Line 2.** Find steady state V and Λ .
- **Line 3.** Read off K^{supplied} .

Outer Tatonnement Loop

```
K = 5.0
for it in 1:max_iter
    env = make_env(hh; aiyagari_prices(K, p)...)
    (; V, Λ, moments) = solve_steady_state_given_env!(hh, env)
    K_supplied = moments.K_supplied
    K_err = abs(K_supplied - K) / K
    K_err <= rtol && break
    K += update_speed * (K_supplied - K)      # damped tatonnement
end
```

Each pass: $\bar{K} \rightarrow R \rightarrow V, \Lambda \rightarrow K^{\text{supplied}} \rightarrow K_{\text{err}}$, then update $\bar{K}$.

Stop when $|K_{\text{err}}|/K < \text{rtol}$.

Comparative statics

Comparative Statics

Change one parameter and re-solve steady state.

Can think of this as the **long-run** response to a shock.

In the example calibration:

- $A = 1 \rightarrow \bar{K}_{\text{pre}} = 5.68, \bar{R}_{\text{pre}} = 0.038.$
- $A = 1.05 \rightarrow \bar{K}_{\text{post}} = 6.14, \bar{R}_{\text{post}} = 0.038.$

Comparative Statics

Comparative statics tells you **long-run** response to shocks.

Doesn't tell you:

- Speed of transition
- Path of transition
- Welfare impacts of shock on those alive at the time.

For these, we need transition dynamics.

Transition Dynamics

Transition Dynamics

- **Steady state.** A single (K, V, Λ) . Time-invariant.
- **Transition.** A sequence $\{(K_t, V_t, \Lambda_t)\}_{t=1, \dots, T}$. The economy is *moving*.

The MIT Shock

A **permanent, unexpected** shock at $t = 1$.

- $t = 0$: At steady state.
- $t = 1$: Parameter permanently changes. Agents are *surprised*.
- $t \geq 1$: *Perfect foresight* of the entire future path of exogenous and endogenous variables.

Example: TFP Shock

Permanent positive TFP shock at $t = 1$:

$$A_t = 1.05 \quad \text{for all } t \geq 1, \quad A_0 = 1.$$

- The *only* exogenous change.
- $\{r_t, w_t, K_t, V_t, \Lambda_t\}$ change in **response**.
- Set $T = 100$ and set $V_{T+1} = V^{\text{new_steady_state}}$ to close the model.

Solving MIT transition

The problem statement

- **Given:** $\{A_t\}_{t=1}^T$.
- **Find:** $\{K_t, V_t, \Lambda_t\}$ such that
 - $V_{T+1} = V_{ss}$ (terminal).
 - $\Lambda_1 = \Lambda_{ss}$ (initial).
 - $V_t = \text{backward}(V_{t+1}, K_t, A_t)$.
 - $\Lambda_{t+1} = \text{forward}(\Lambda_t, V_{t+1})$.
 - Markets clear: $K_t = K_t^{\text{supplied}}$.

Algorithm

1. Guess path $\{K_t\}$.
2. Iterate V backward.

$$V_{T+1} \leftarrow V_{\text{ss}^{\text{Post}}}, \quad V_t = \text{backward}(V_{t+1}, K_t, A_t).$$

3. Simulate Λ forward.

$$\Lambda_1 \leftarrow \Lambda_{\text{ss}^{\text{Pre}}}, \quad \Lambda_{t+1} = \text{forward}(\Lambda_t, V_{t+1}).$$

4. Update K_{path} .

$$K_t \leftarrow (1 - s) K_t + s K_t^{\text{supplied}}.$$

Note: Rootfinding

Ultimately, the problem is to find the root of the excess demand function in sequence space:

$$\text{excess_demand}(\{K_t\}) = \vec{0}.$$

For demonstration, we are using the simple and robust method of tatonnement.

Gradient-based methods using the sequence-space Jacobian can be much faster.

The algorithm

```
hh_path = [aiyagari_household(p_new) for _ in 1:T] # one chain per period
K_path = fill(K_ss, T) # initial guess
V_path[T+1] .= V_ss; Λ_path[1] .= Λ_ss # boundary conditions
while res > tol
    env_path = [make_env(hh_path[t]; aiyagari_prices(K_path[t], p_new)...) for t in 1:T]
    for t in T:-1:1 # backward sweep
        copyto!(V_path[t], backward!(hh_path[t], V_path[t+1], env_path[t]))
    end
    for t in 1:T # forward sweep
        copyto!(Λ_path[t+1], forward!(hh_path[t], Λ_path[t]))
        K_supplied[t] = compute_moments(hh_path[t], Λ_path[t+1], env_path[t]).K_supplied
    end
    res = maximum(abs, K_supplied .- K_path)
    K_path .= (1 - update_speed) .* K_path .+ update_speed .* K_supplied # damped tatonnement
end
```

Implementation

Initial Steady State

We need both:

- V_{ss} : terminal for backward sweep.
- Λ_{ss} : initial for forward sweep.
- Section 2's tatonnement gives both.
- **Setup:** $V_{ss} \rightarrow V_{\text{path}}[T+1]$; $\Lambda_{ss} \rightarrow \Lambda_{\text{path}}[1]$; initial $\{K_t\} \equiv K_{ss}$.

Backward sweep

```
for t in T:-1:1
    env_t = make_env(hh_path[t]; aiyagari_prices(K_path[t], p_new)...
    copyto!(V_path[t], backward!(hh_path[t], V_path[t+1], env_t))
end
```


Forward sweep (re-seat kernels!)

```
for t in 1:T
    env_t = make_env(hh_path[t]; aiyagari_prices(K_path[t], p_new)...
    copyto!(  $\Lambda$ _path[t+1], forward!(hh_path[t],  $\Lambda$ _path[t]))
    K_supplied[t] = compute_moments(hh_path[t],  $\Lambda$ _path[t+1], env_t).K_supplied
end
```

Outer tatonnement update

```
res = maximum(abs, K_supplied .- K_path)
K_path .= (1 - update_speed) .* K_path .+ update_speed .* K_supplied
```

IRF: capital

Shape.

- ΔA raises r .
- Higher $r \rightarrow$ more savings.
- More saving $\rightarrow$ higher $K \rightarrow$ lower r . Self-correcting.

Lag. Wealth distribution has inertia. Peak in K lags peak in A by ~ 5 periods.

IRF: r_t and w_t

r_t .

- Jumps on impact (A_1 before K moves).
- Falls below SS as K overshoots.
- Returns to r_{ss} as $A_t \rightarrow 1$.

w_t .

- Same pattern, opposite sign.
- Above SS while K overshoots.

Welfare

- Wealthy agents: transient capital gain on r .
- Wage-dependent agents: pick up the w side.

For Aiyagari we get a *distribution* of welfare effects, period by period. Disentangling the two channels is one thing MIT shocks let you do.

Pick T large enough

- T = horizon where $A_T \approx 1$ and $V_{T+1} = V_{ss}$.
- Too small $\rightarrow$ terminal condition contaminates the path.

Rule of thumb. T such that $A_T - 1 < 10^{-3}$. At $\rho = 0.85$, $A_0 = 1.05$: $T \approx 25$. We use $T = 100$.